

Information Retrieval

section (1)

What Types of Information?

Text (Documents)

XML and structured documents

Images

Audio

Video

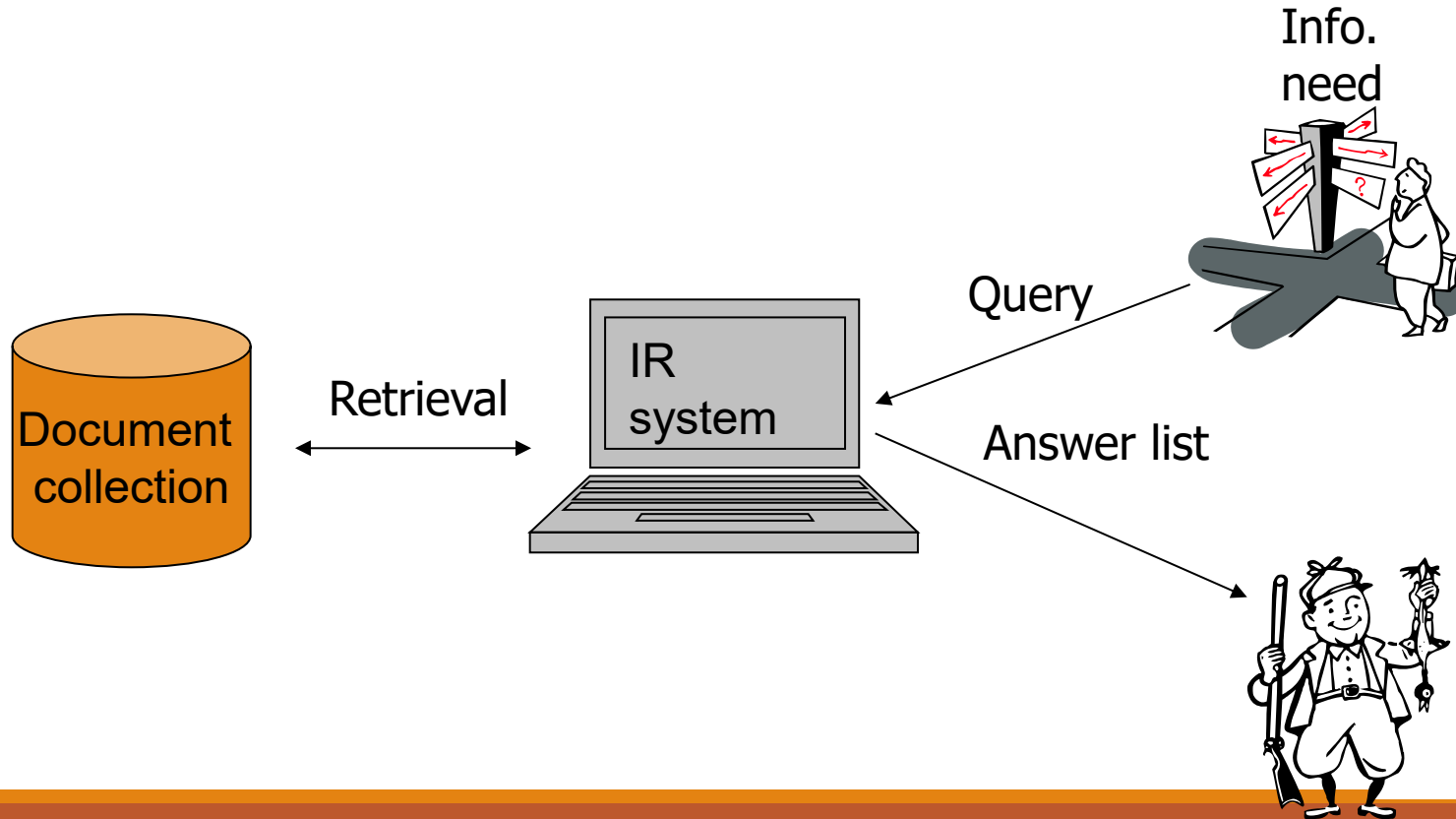
Source code

Applications/Web services

A solid orange horizontal bar at the bottom of the slide.

Goal of IR

Goal = find documents *relevant* to an information need from a large document set



- عندك مكتبة ضخمة جدًا مليانة كتب (ده هو large document set).
- طالب محتاج يعرف "فوائد الذكاء الاصطناعي في الطب" (ده هو information need).
- الطالب يكتب استعلام (query) زي: "AI in healthcare benefits".
- نظام استرجاع المعلومات (IR system) يبحث في المكتبة ويرجع له:
- بحث علمي عن "استخدام الذكاء الاصطناعي في تشخيص الأمراض"
- مقالة عن "فوائد الذكاء الاصطناعي في تحليل صور الأشعة"
- تقرير عن "الذكاء الاصطناعي وإدارة السجلات الطبية"

Main problems in IR

1. Document and Query Indexing

- **Problem:** How to best represent the content of documents and user queries so that the system can search efficiently?
- **Example:**
 - Document: *"Artificial Intelligence improves healthcare"*
 - Query: *"AI in hospitals"*
 - Should the system understand that **AI = Artificial Intelligence**?
- The challenge is choosing a good representation (keywords, vectors, semantic meaning, etc.).

Main problems in IR

2. Query Evaluation (Retrieval Process)

- **Problem:** To what extent does a document correspond to a query?
- How should the system measure the **relevance** between a query and a document?
 - Simple word overlap?
 - Weighted scoring (e.g., TF-IDF)?
 - Advanced models (e.g., embeddings, BERT)?
- **Example:**
 - Query: *"AI in healthcare"*
 - Document A: *"Artificial Intelligence is used in hospitals"* → Highly relevant.
 - Document B: *"Artificial Intelligence in self-driving cars"* → Not relevant.

Main problems in IR

3. System Evaluation

- **Problem:** How to measure how good the system is?
- Two key metrics are used:
 - **Precision:**
 - Are the retrieved documents relevant?
 - Example: The system returns 10 results, 7 are correct → Precision = 70%.
 - **Recall:**
 - Are all relevant documents retrieved?
 - Example: There are 20 relevant documents in the collection, but the system finds only 15 → Recall = $15/20 = 75\%$.
- **Challenge:**
 - Sometimes high precision but low recall (few but accurate results).
 - Sometimes high recall but low precision (many results, but many irrelevant).



What is Different about IR from the rest of Computer Science

- **Traditional Computer Science problems:** one precise, correct answer.
Examples:
 1. Sort the following 10 integers
 - Input: [10, 3, 7, 1, 5, 9, 2, 6, 8, 4]
 - Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
- **Information Retrieval problems:** multiple possible answers, ranked by relevance.

Content

- Tokenization
- Stop word removal
- Stemming
- Term – document Matrix
- Inverted index
- TF-IDF

Text pre-processing

Need to perform it on both:

- Collection of documents
- Query

→ Tokenization – Stop words – Stemming

Stop words / Stop list

-
- function words do not bear useful information for IR

Ex: of, in, about, with, I, although, ...

Stop list: contain stop words, not to be used as index

- **Prepositions** (e.g., "on," "in," "at")
- **Pronouns** → "he," "she," "it," "they"
- Some adverbs and adjectives
- Some frequent words (e.g. document) → "document," "information," "search"

Why do we need to remove stop words?

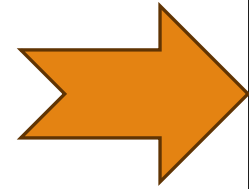
- Reduce indexing (or data) file size
- stop words accounts 20-30% of total word counts.
- **Improve efficiency**
 - stop words are not useful for searching or text mining
 - stop words always have a large number of hits

Tokenization.

-
- A document is converted to a stream of tokens, e.g. individual words.
 - Tokenization without libraries
 - Tokenization with `nltk`
 - Tokenization with `spacy`

Tokenization without libraries

```
1 txt="this is a text full of content and we need to clean it up"
2 words=txt.split(" ")
3 for w in words:
4     print(w)
```



```
this
is
a
text
full
of
content
and
we
need
to
clean
it
up
```

Download python and vs

<https://www.youtube.com/watch?v=8vW2aGmpgCs>

Download pip install

<https://www.youtube.com/watch?v=4d1xO6xcBzw>

- **cmd**
 - `curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py`
 - `python get-pip.py`

What is nltk??

- **NLTK** = Natural Language Toolkit
- It is a Python library for working with Natural Language Processing (NLP).
- It helps developers analyze and process text without building everything from scratch.
- Download nltk
 - `pip install nltk` → in cmd https://www.youtube.com/watch?v=yqynaL3B_Mc
 - `import nltk`
 - `nltk.download()`

nlTK Downloader

```
C:\Users\User>pip install nltk
Requirement already satisfied: nltk in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (3.9.1)
Requirement already satisfied: click in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from nltk) (8.3.0)
Requirement already satisfied: joblib in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from nltk) (1.5.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from nltk) (2025.9.18)
Requirement already satisfied: tqdm in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: colorama in c:\users\user\appdata\roaming\python\python313\site-packages (from click->nltk) (0.4.6)

C:\Users\User>pip show nltk
Name: nltk
Version: 3.9.1
Summary: Natural Language Toolkit
Home-page: https://www.nltk.org/
Author: NLTK Team
Author-email: nltk.team@gmail.com
License: Apache License, Version 2.0
Location: C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: click, joblib, regex, tqdm
Required-by:
```


nltk Downloader

```
1 import nltk
2 nltk.download()
```

NLTK Downloader

File View Sort Help

Collections Corpora Models All Packages

Identifier	Name	Size	Status
maxent_ne_chunker	ACE Named Entity Chunker (Maximum entropy)	12.8 MB	installed
maxent_treebank_pos	Treebank Part of Speech Tagger (Maximum entropy)	9.7 MB	installed
moses_sample	Moses Sample Models	10.5 MB	installed
mwa_ppdb	The monolingual word aligner (Sultan et al. 2015)	1.5 MB	installed
perluniprops	perluniprops: Index of Unicode Version 7.0.0 characters	97.9 KB	installed
porter_test	Porter Stemmer Test Files	195.8 KB	installed
punkt	Punkt Tokenizer Models	13.3 MB	installed
rsdp	RSLP Stemmer (Removedor de Sufixos da Lingua Portuguesa)	3.7 KB	installed
sample_grammars	Sample Grammars	19.8 KB	installed
snowball_data	Snowball Data	6.5 MB	installed
spanish_grammars	Grammars for Spanish	4.0 KB	installed
tagsets	Help on Tagsets	33.7 KB	installed
universal_tagset	Mappings to the Universal Part-of-Speech Tagset	18.6 KB	installed
vader_lexicon	VADER Sentiment Lexicon	88.4 KB	installed
wmt15_eval	Evaluation data from WMT15	374.1 KB	installed
word2vec_sample	Word2Vec Sample	47.1 MB	installed

Download Refresh

Server Index: https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.html

Download Directory: E:\programs\anaconda\NLTK-SETUP

Finished downloading collection 'all-nltk'.

NLTK Downloader

File View Sort Help

Collections Corpora Models All Packages

Identifier	Name	Size	Status
all	All packages	n/a	installed
all-corpora	All the corpora	n/a	installed
all-nltk	All packages available on nltk_data gh-pages branch	n/a	installed
book	Everything used in the NLTK Book	n/a	installed
popular	Popular packages	n/a	installed
tests	Packages for running tests	n/a	installed
third-party	Third-party data packages	n/a	installed

Download Refresh

Server Index: https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.html

Download Directory: E:\programs\anaconda\NLTK-SETUP

Finished downloading collection 'all-nltk'.

Tokenization with nltk

```
1 from nltk.tokenize import word_tokenize
2 text="how are you? i'am fine."
3 print(word_tokenize(text))
```

```
['how', 'are', 'you', '?', "i'am", 'fine', '.']
```

Tokenization with spacy

in cmd

- `pip install spacy`
- `python -m spacy download en_core_web_sm`

in anaconda prompt

- `conda install -c conda-forge spacy-model-en_core_web_sm`

Tokenization with spacy

```
1 #Tokenization with spacy
2 import spacy
3 nlp=spacy.load("en_core_web_sm")
4 doc=nlp('Tesla is looking at buying U.S. startup for $6 million')
5 for token in doc:
6     print(token.text, "\t is alpha", token.is_alpha, "is stop=", token.is_stop)
```

Tesla	is alpha True	is stop= False
is	is alpha True	is stop= True
looking	is alpha True	is stop= False
at	is alpha True	is stop= True
buying	is alpha True	is stop= False
U.S.	is alpha False	is stop= False
startup	is alpha True	is stop= False
for	is alpha True	is stop= True
\$	is alpha False	is stop= False
6	is alpha False	is stop= False
million	is alpha True	is stop= False

Remove stop words

```
stopwords=['this' , 'that' , 'and' 'we' , 'it' , 'to' , 'is' , 'of' , 'up','need ']  
text="this is a text full of content and we need to clean it up"  
words=text.split(" ")  
showtlist_words=[]  
for w in words:  
    if w not in stopwords:  
        showtlist_words.append(w)  
    else:  
        continue  
print("oeriginal text=",text)  
print("sentence with stop words removed",' '.join(showtlist_words))
```

```
oeriginal text= this is a text full of content and we need to clean it up  
sentence with stop words removed a text full content and we need clean
```

Remove stop words with nltk

```
1 ~ import nltk
2  from nltk.tokenize import word_tokenize
3  stopwords=nltk.corpus.stopwords.words('english')
4 ~ def remove_stopwords(txt):
5      words=word_tokenize(txt)
6      clean=[]
7 ~     for word in words:
8 ~         if word not in stopwords:
9             clean.append(word)
10 ~        else:
11            continue
12     return clean
13 text="this is a text full of content and we need to clean it up"
14 print(remove_stopwords(text))
```

```
['text', 'full', 'content', 'need', 'clean']
```